

Listas Ligadas o simplemente ligadas

Estructura de Datos y Algoritmos

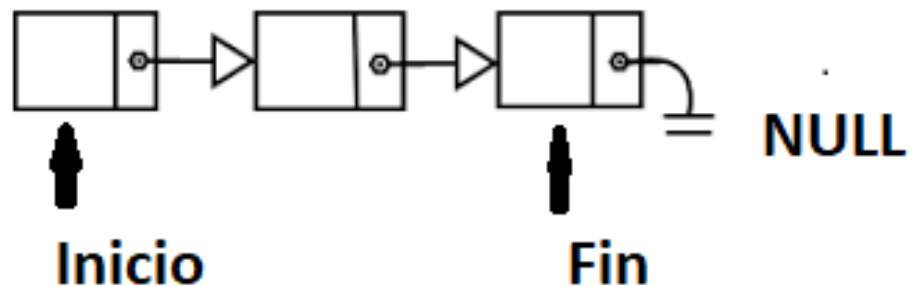
Profesora:

Elba Karen Sáenz García



Lista

- Estructura de datos lineal compuesta por una secuencia de cero o más elementos de un tipo determinado y ordenados de alguna forma.
- Puede crecer o disminuir en el número de elementos y podrán insertarse o eliminarse elementos en cualquier posición sin alterar su orden lógico



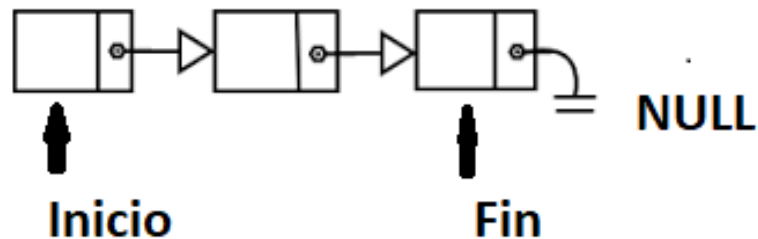
Operaciones Listas

- Algunas de las operaciones que se pueden realizar sobre las listas son las siguientes:
- $\text{inserta}(x,p,L)$: Inserta x en la posición P de la lista L .
- $\text{localiza}(x,L)$: Obtiene la posición P de x en la lista L .
- $\text{recupera}(P,L)$: Devuelve el elemento que está en la posición P de la lista L .
- $\text{suprime}(P,L)$ Elimina el elemento en la posición P de L

Operaciones Listas

- siguiente(P,L) y anterior(P,L): Devuelven la posición P siguiente u anterior respectivamente de L.
- Anula (L): Ocasiona que L se convierta en lista vacía y devuelva la posición final FIN(L).
- Primero(): Esta función devuelve la primera posición de la lista , si L esta vacía la posición que regresa es FIN (L).
- Imprime Lista(L):Imprime los elementos de L en orden de aparición de la lista

Listas simplemente ligadas



- Se seguirán utilizando las estructuras para la definición de un nodo y el control de lista

Dato	Siguiente
------	-----------

```
#include <stdio.h>
#include <stdlib.h>

struct NodoLista{
    int dato;
    struct NodoLista *siguiente;
};

typedef struct NodoLista nodo;

struct controlLista{
    nodo *inicio;
    nodo *fin;
    int numElem;
} ;

typedef struct controlLista Lista;
```

Crear o inicializar la lista

```
void inicializacion(Lista *lista){  
    lista->inicio = NULL;  
    lista->fin = NULL;  
    lista->numElem = 0;  
}
```

```
int main(){  
    Lista lista1;  
  
    inicializacion(&lista1);  
  
    return 0;  
}
```

Si está Vacía

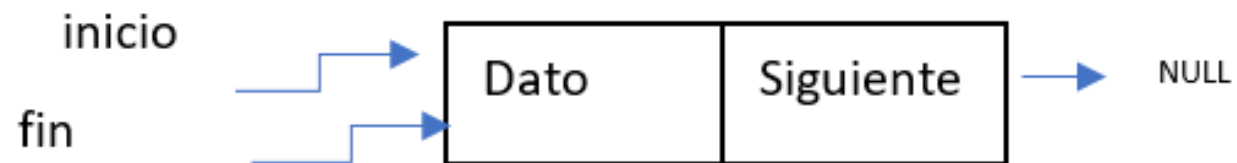
```
int vacia(Lista *lis){  
    if(lis->inicio==NULL && lis->fin==NULL && lis->numElem==0)  
    {  
        return 0;  
    }  
    else  
    {  
        return 1;  
    }  
}
```

Inserción

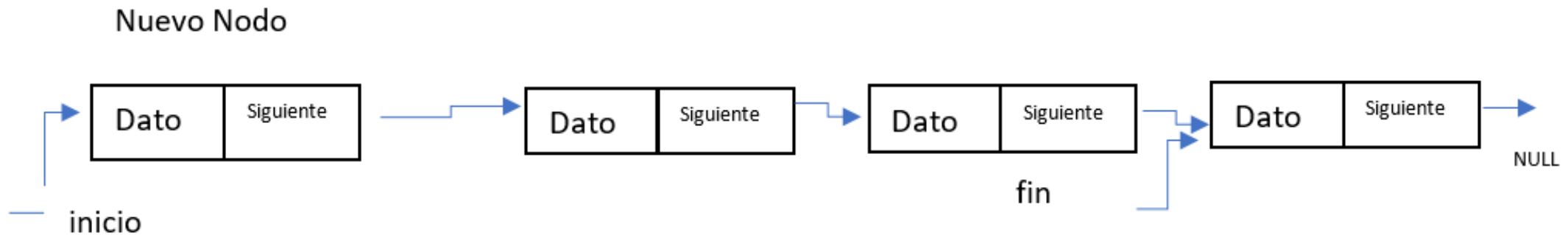
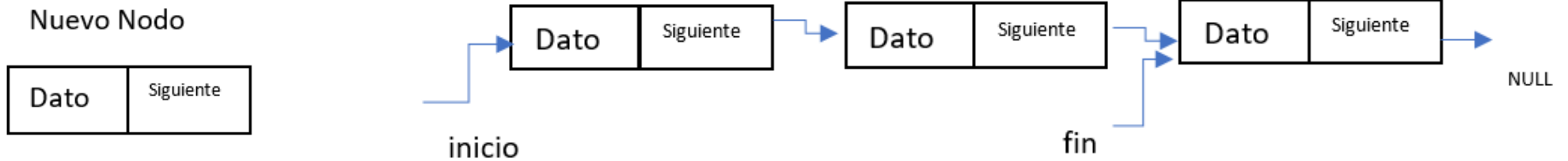
- Para la inserción se analiza cuando se quiere insertar al inicio, al final y después de una posición dada.

Insertar al inicio

- se considera en el planteamiento que pasa cuando está vacía y cuando no.
- Si **está vacía los apuntadores inicio y fin apuntan a NULL**. Entonces se forma el nodo nuevo a insertar y después las variables apuntador inicio y fin deben apuntar al nuevo nodo.



Insertar al inicio con datos ya en la lista



incrementar el numero de elementos de la lista.

```

int insertarInicio (Lista *lis, float dato){
    nodo *nuevoNodo;

    nuevoNodo=creaNodo(dato);
    if(nuevoNodo==NULL){
        return 1;
    }

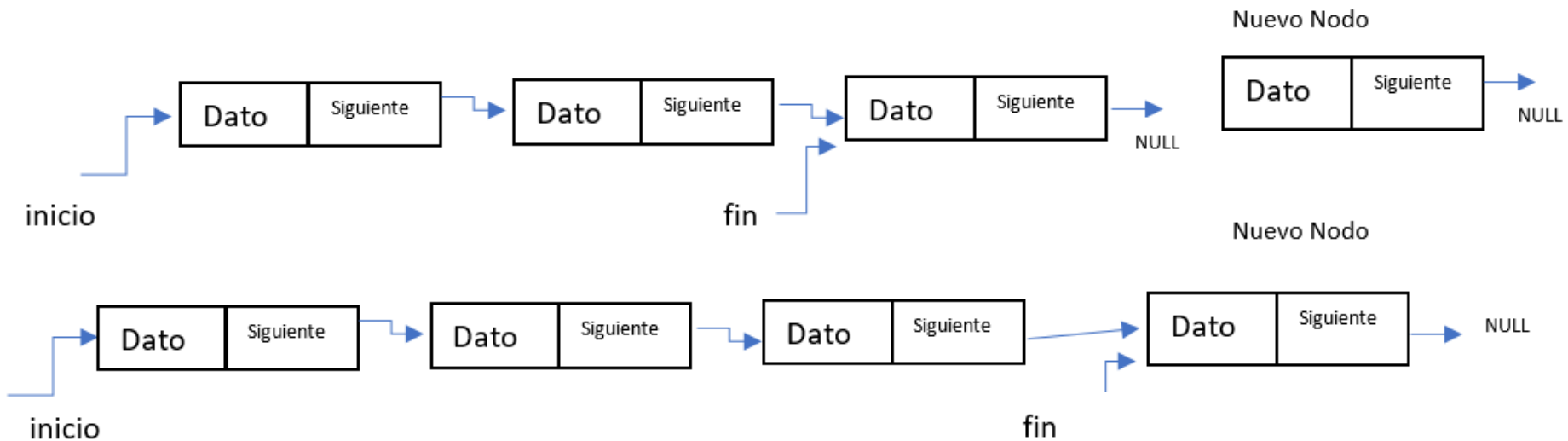
    if( !vacía(lis)){
        nuevoNodo->siguiente=NULL;
        lis->fin=nuevoNodo;
        lis->inicio=nuevoNodo;
    }
    else{
        nuevoNodo->siguiente=lis->inicio;
        lis->inicio=nuevoNodo;
    }
    lis->numElem++;
    return 0;
}

```

Insertar al final

- Se puede insertar si esta vacía o si no lo está, en el primer caso es igual al explicado en insertar al inicio.

Insertar si no está vacía.



```
int insertarFinal(Lista *lis, float dato){
    nodo *nuevoNodo;

    nuevoNodo=creaNodo(dato);
    if(nuevoNodo==NULL){
        return 1;
    }
    if( !vacía(lis)){
        lis->fin=nuevoNodo;
        lis->inicio=nuevoNodo;
    }
    else{
        lis->fin->siguiente=nuevoNodo;
        lis->fin=nuevoNodo;
    }
    lis->numElem++;
    return 0;
}
```

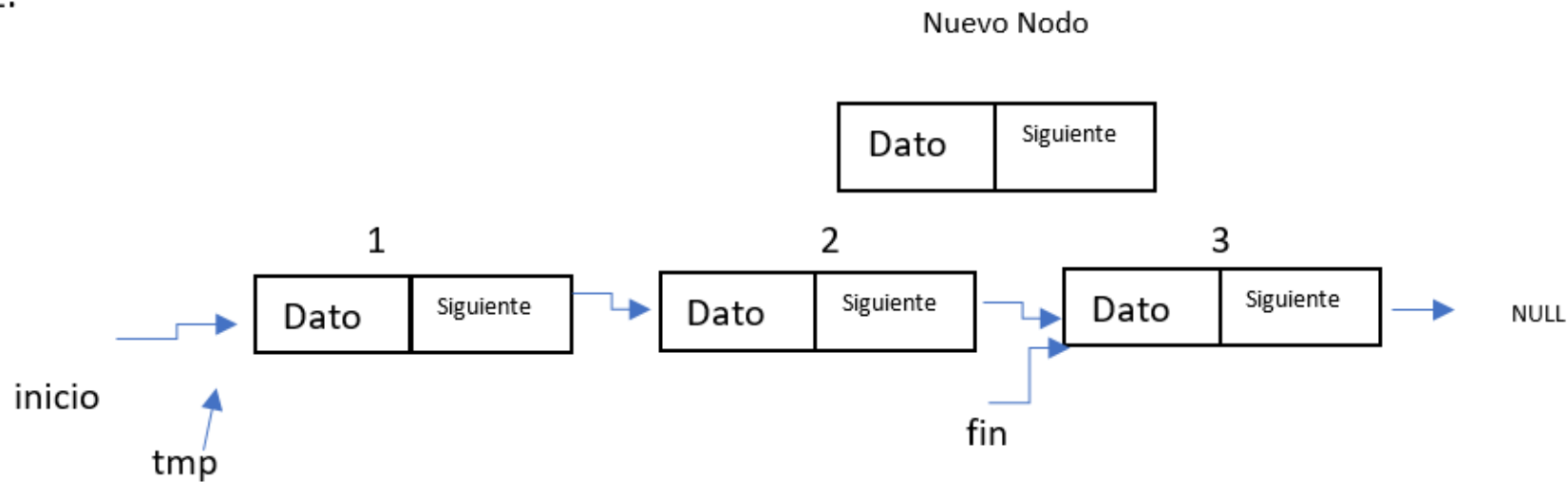
Inserción después de una posición dada

- La inserción necesita una posición que no sea inicio o fin
- La posición será un número entero y el nuevo Nodo se inserta después de esa posición.
- se utiliza un apuntador a nodo (tmp) que irá tomando las direcciones desde el nodo del inicio hasta el nodo con la posición dada.

Ejemplo 1

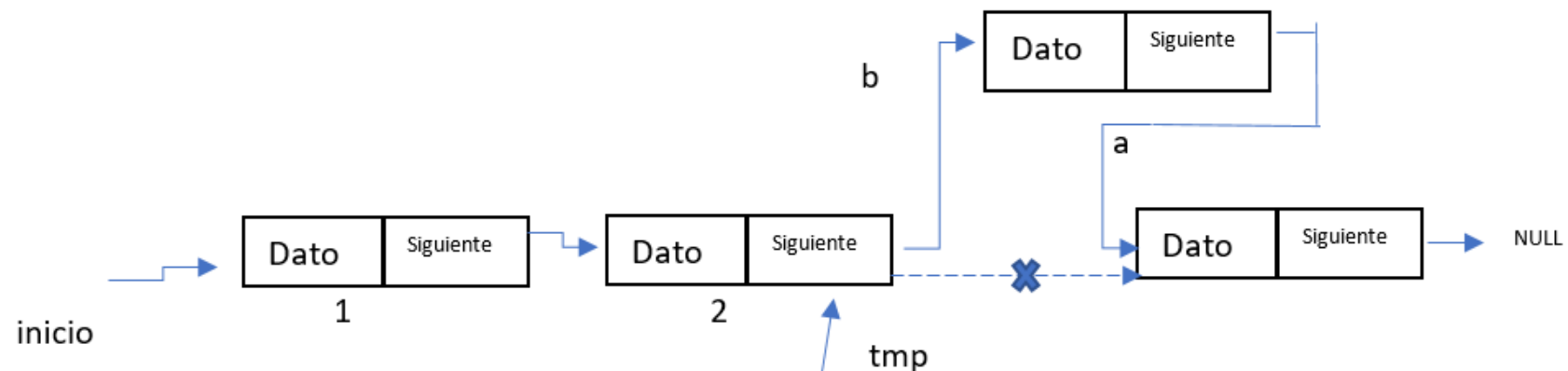
- Se insertará el nodo después de la posición 2, antes tmp se moverá desde inicio hasta a la posición 2.

2.



Continuación de ejemplo 1

- Si tmp está en la posición 2 se realizan los siguientes enlaces.
 - El siguiente del nuevo nodo apuntará a lo que apunta el siguiente del nodo al que apunta tmp
 - El siguiente del nodo al que apunta tmp apunta a nuevo nodo **nuevoNodo**;




```

int insertarPos(Lista *lis, float dato, int pos ){
int i;
nodo *tmp, *nuevoNodo;
    if(pos < 0 || pos > lis->numElem){ // Si la posición dada es menor a cero y mayor al # de elementos
        printf("esa posicion no existe debe ser de a a %d", lis->numElem);
        return -1; }
    nuevoNodo=creaNodo(dato);
    if(nuevoNodo==NULL){
        return 1;
    }
    if (pos==0) { //si la posicion dada es el inicio
        printf("\n usar insertar al inicio ");
        return -1; }

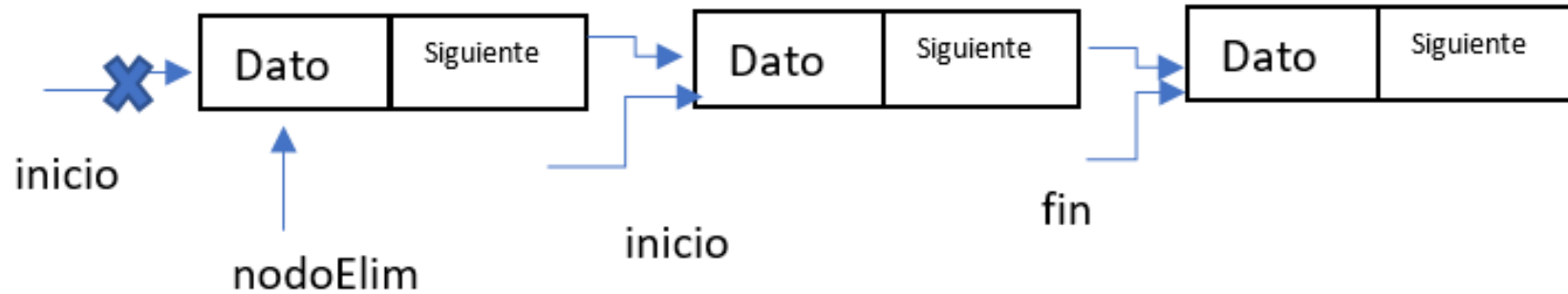
    tmp=lis->inicio; //iniciar tmp en el inicio de lista
    for(i=1;i<=pos;i++){ //mover tmp a la posicion dada
        tmp=tmp->siguiente;
    }
    if(tmp->siguiente==NULL){ // Si la posicion dada es el final
        printf("\n usar insertar al final");
        return -1;
    }
    // inserción del nodo en la lista
    nuevoNodo->siguiente=tmp->siguiente;
    tmp->siguiente=nuevoNodo;
    lis->numElem++;
    return 0;
}

```

Eliminar un Nodo

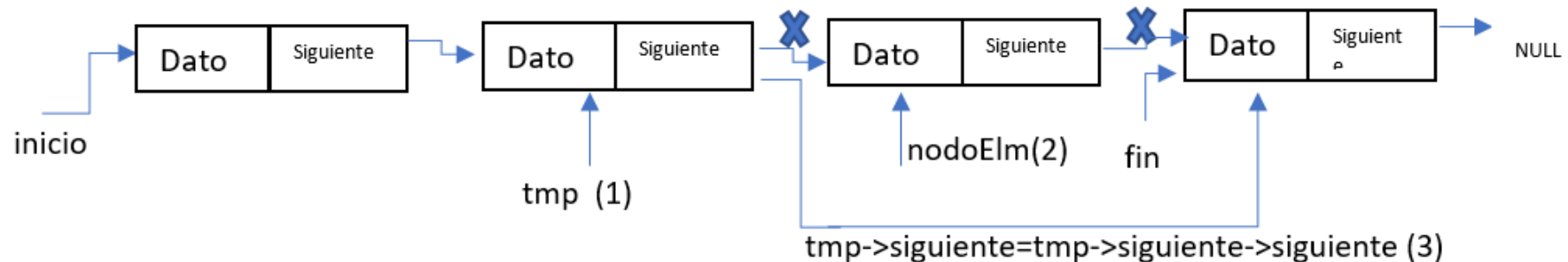
- Para eliminar un nodo en el análisis se considera eliminar al inicio y después de una posición.
- Para el caso de la eliminación del último se deberá proporcionar la posición del penúltimo elemento

Eliminar al inicio



Eliminar después de una posición

- (1) Se sitúa un apuntador a nodo **tmp** en la posición proporcionada
- (2) se coloca un apuntador **nodoElim** en el nodo siguiente de la posición (el que se eliminará)
- (3) $tmp \rightarrow siguiente = tmp \rightarrow siguiente \rightarrow siguiente$



Eliminar un Nodo

```
float eliminarPos(Lista *lis, int pos){
    nodo *tmp,*nodoElim; //apuntadores a nodo
    float dato;
    int i;

    if(!vacía(lis)){ //si lista vacía no elimina
        printf("\n lista vacía");
        return -1.0;
    }
    if(pos < 0 || pos > lis->numElem) { //si la posición no es válida, termina
        printf ("\nposición no válida");
        return -2;
    }
    if (pos==0) { // Si se quiere eliminar al inicio, la posición dada debe ser 0 y elimina la posición 1

        if(lis->inicio==lis->fin) //Si la lista tiene un elemento, al eliminar la lista queda vacía y toma valores iniciales
            inicialización(lis);
        else { //Si se elimina del inicio y la lista tiene más de un elemento

            nodoElim=lis->inicio; //colocamos nodoElim en el inicio
            lis->inicio=lis->inicio->siguiente; //inicio se mueve al siguiente
        }
    }
}
```

```

else{ // Se elimina después de una posición mayor o igual a 1

    tmp=lis->inicio; //se coloca tmp al inicio
    for(i=1;i<pos;i++) // se mueve tmp hasta la posición
        tmp=tmp->siguiente;

    if(tmp->siguiente==NULL){
        printf("\nLa posición dada es del último elemento, para borrarlo colocar posición del penúltimo");
        return -3;
    }

    nodoElim=tmp->siguiente; // el apuntador nodoElim se coloca en el siguiente de tmp
    tmp->siguiente=tmp->siguiente->siguiente; // se quita la liga con el nodo a eliminar

}
lis->numElem--; //Se decremanta el tamaño de la lista

dato=nodoElim->dato; // se recupera el dato
free(nodoElim); // Se libera memoria del nodo eliminado de la lista
return dato;
}

```

Mostrar

```
void mostrar( Lista *lis ){
    nodo *tmp;
    tmp=lis->inicio;
    while(tmp!=NULL){
        printf("\n dato = %f ",tmp->dato);
        tmp=tmp->siguiente;
    }
}
```

```

int main(){
    Lista lista1;
    float dato;

    inicializacion(&lista1);

    insertarInicio (&lista1, 5.7);
    printf("\n Dato  1 inseratdo");
    mostrar( &lista1  );

    insertarInicio (&lista1, 6.7);
    printf("\n Dato  3 inseratdo");
    mostrar( &lista1  );

    insertarFinal(&lista1, 7.8);
    printf("\n Dato  3 inseratdo");
    mostrar( &lista1  );

    insertarPos(&lista1, 9.9,2);
    printf("\n Dato  4 inseratdo");
    mostrar( &lista1  );

    dato=eliminarPos(&lista1,1);
    printf("\n dato eliminado %f",dato );
    mostrar( &lista1  );

    dato=eliminarPos(&lista1,2);
    printf("\n dato eliminado %f",dato );
    mostrar( &lista1  );

    return 0;
}

```